Simulacro 0 — Carrera de tortugas

Jueves 29 de octubre de 2026 · aula de teoría (A0.12) · 40 minutos · NO PUNTÚA

> **Para practicar en casa.** Es el enunciado que se hizo en clase el 29 de octubre. Hazlo en 40 minutos, sin apuntes ni IA, y compara con tu informe de corrección de la Enseñanza Virtual. No hay que entregarlo: las instrucciones de entrega son las del día del simulacro y son las mismas que tendrás en el control.
>
> Durante el simulacro no se permite utilizar asistentes de IA generativa. Debes trabajar con los mismos recursos autorizados que se indicarán para los controles.

Este ejercicio **no cuenta para la nota**. Sirve para dos cosas:

1. Que pases por el procedimiento completo de un control —la plantilla, `identificacion.md`, el `.zip`, la subida— cuando equivocarse todavía sale gratis.
2. Que compruebes **qué sabes escribir tú**, sin ayuda, a finales de octubre. Los dos controles que sí puntúan son el Control 1 y el Control 2, en el laboratorio.

Recibirás tu informe de corrección igual que en un control real.

Antes de empezar (en tu portátil)

1. Descarga el fichero `simulacro0.zip` y descomprímelo.
2. Abre **la carpeta descomprimida** en VS Code, no un fichero suelto: el CSV se lee con la ruta relativa `data/carreras.csv`.
3. Cierra cualquier asistente de IA (Copilot, chat, extensiones). Sólo puedes tener abiertos VS Code, la terminal y la Enseñanza Virtual.

Entrega

1. Rellena `identificacion.md` con tu nombre, UVUS y DNI. Una entrega sin identificar no se corrige.
2. Implementa las cuatro funciones de `simulacro.py`. **No cambies sus nombres, sus parámetros ni los campos de `Participacion`**: el corrector los usa tal cual.
3. Comprime la carpeta en `apellidos_uvus.zip` y súbelo a la tarea «Simulacro 0» de la Enseñanza Virtual **antes de que acabe el tiempo**.

> **Si tu módulo no se puede importar** —un `SyntaxError`, o código suelto que falla al cargarse— **no se ejecuta ninguna función y la nota es 0**, aunque el resto esté bien. Antes de entregar, ejecuta

`simulacro.py` una vez y comprueba que no da error. Es lo mismo que pasará en los controles y en enero: mejor descubrirlo hoy.

El formato

Es **el mismo que el del Control 1 y el del examen de enero**, en pequeño: un fichero CSV, una `NamedTuple` obligatoria y unas funciones que leen, filtran y agregan esos datos. El tema es el de tu **M1**: la carrera de tortugas. En el Control 1 el formato será este mismo, sobre el Arkanoid del M2.

Los datos

`data/carreras.csv` guarda los resultados de varias carreras de tortugas, en UTF-8 y con cabecera. Cada línea es una tortuga en una carrera: el nombre de la carrera, la longitud de su pista, el nombre de la tortuga y su velocidad. **No está ordenado por carrera.**

```
carrera,distancia,tortuga,velocidad
jardin,620,roja,10
playa,400,roja,40
jardin,620,azul,12
...
```

Las carreras siguen las reglas de tu M1:

- En cada vuelta, cada tortuga avanza tantas unidades como su `velocidad`, y llega a la meta en cuanto alcanza **o supera** la `distancia`.
- Una tortuga con velocidad 0 o negativa no llegaría nunca: **no corre**.
- Si varias tortugas llegan en **la misma vuelta**, es un **empate**, aunque sus velocidades sean distintas.

NamedTuple obligatoria

Ya viene escrita en la plantilla:

```python
Participacion = NamedTuple("Participacion", [
    ("carrera", str),
    ("distancia", int),
    ("tortuga", str),
    ("velocidad", int),
])
```

Ejercicios

1. `lee_participaciones(ruta) -> list[Participacion]` · 2,5 puntos

Lee el CSV de `ruta` y devuelve la lista de participaciones, en el mismo orden en que aparecen en el fichero. Abre el fichero con `encoding="utf-8"`, sáltate la cabecera y convierte `distancia` y `velocidad` a enteros.

```
>>> lee_participaciones("data/carreras.csv")[0]
Participacion(carrera='jardin', distancia=620, tortuga='roja', velocidad=10)
```

2. `vueltas_hasta_meta(velocidad, distancia) -> int` · 2,0 puntos

Número de vueltas que tarda una tortuga en alcanzar o superar `distancia` si en cada vuelta avanza `velocidad`. Lanza **`ValueError`** si la velocidad no es positiva.

```
>>> vueltas_hasta_meta(10, 620)
62
>>> vueltas_hasta_meta(12, 620)   # con 51 vueltas sólo llega a 612
52
```

3. `ganadora(participaciones, carrera) -> str` · 3,0 puntos

Nombre de la tortuga que llega a la meta de `carrera` en menos vueltas. Si varias llegan en esa misma vuelta, devuelve `"empate"`. Las tortugas que no corren no cuentan. Lanza **`ValueError`** si en esa carrera no corre ninguna tortuga, también si la carrera no existe.

Usa `vueltas_hasta_meta`.

```
>>> ganadora(lee_participaciones("data/carreras.csv"), "jardin")
'azul'
```

4. `victorias(participaciones) -> dict[str, int]` · 2,5 puntos

Devuelve un diccionario que asocia a cada tortuga el número de carreras que ha ganado. Los empates no cuentan como victoria para nadie, y sólo aparecen las tortugas con al menos una victoria. Puedes suponer que en cada carrera corre al menos una tortuga.

> Primero reúne los nombres de las carreras **sin repetir** (el fichero no está agrupado). Después llama a `ganadora` con cada una y ve sumando en el diccionario: antes de sumar en `resultado[tortuga]`, comprueba si la tortuga ya está.

Cómo se corrige

Igual que en los controles: cada ejercicio se prueba con varios casos (el general, los empates, las velocidades no válidas y la lista vacía) y tu nota en cada uno es su peso multiplicado por la fracción de casos que superas: **una función a medias puntúa**. Los ejercicios 3 y 4 se prueban con datos correctos aunque tu `lee_participaciones` falle.